Report – Exam Scheduling

Hoang Le | Ian Cornin

Dialovos/exam-scheduling-algos-analyze

To start, we implemented the code with assistance from GenAI, and now we're working on optimization by identifying the optimal parameters for each algorithm. The basic strategy has been fine-tuned to perform well on a local machine. There are still some slight performance improvements to be made, so we'll set up an auto-tuner to iterate over many parameters in Google Colab and import the results back. That is not the main reason for doing so; the main reason is to run out of time on brute-force algorithms, as they are order-of-magnitude slower for larger set sizes. With that being said, we have the result that we needed, and we could stop here, but we ask ourselves if it could be better, and if it could, then can we do it? We analyzed the floor runtime for algos and tried to push toward that while maintaining solution quality.

Understanding and defining the problem are important. This would determine how we solve the problem and what key measures to take. We went with the Second International Timetabling Competition (ITC-2007) definition. Having a feasible solution (no hard constraint violations) matters more than having an infeasible solution. Soft constraint penalties are by-products that we aim to reduce in the process.

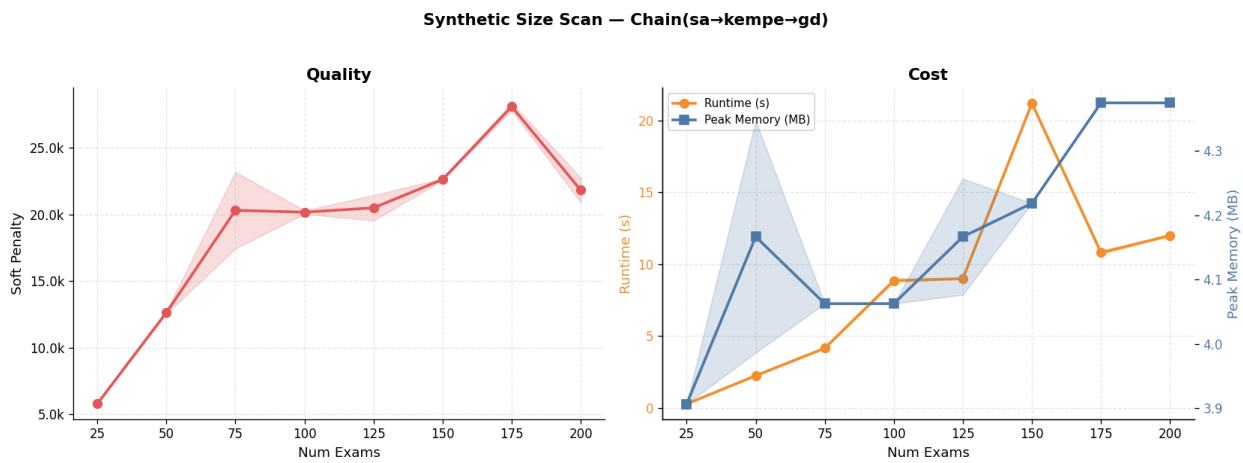
There were some roadblocks during our initial implementation, and one of them was the language we used. Python is two orders of magnitude slower than other lower-level languages such as C++. The stopping point was choosing between C++ and Rust, and I decided to go with Rust since it is a lot harder to implement, and there would be some issues that would be difficult to solve. Since notebooks have been set up and others, rebuilding from the ground up would mean a larger loss of progress for us. The solution I can come up with is to build a bridge that compiles the algorithm in C++ via a subprocess and returns the result in the same format back to Python. The translation between two languages would slow down, so it would only make sense to use the same setup in Python as in C++ and benchmark it there before returning the result.

Now that the language speed has been solved, what is our next roadblock? It boils down to picking the right strategy for each algorithm. Here is a comprehensive table for each strategy.

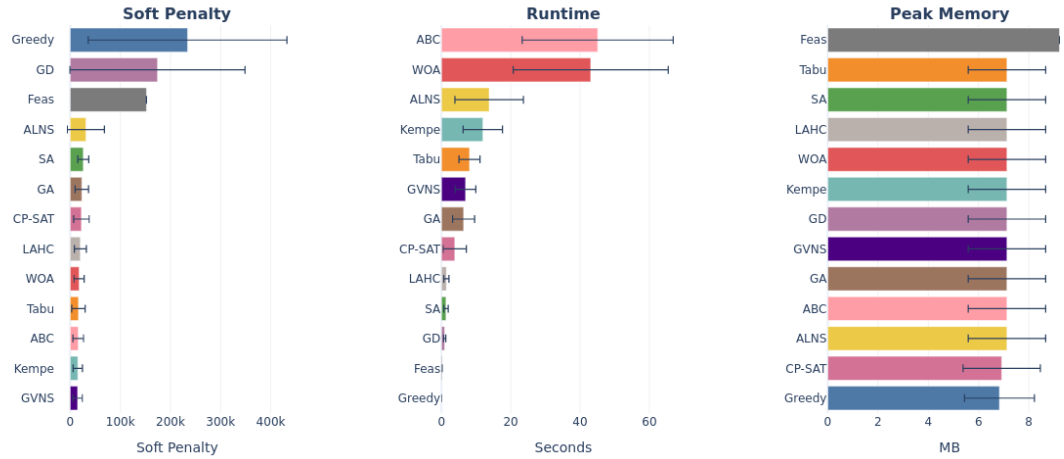
Strategy	**Algos that Use it**	**Impact**
Delta evaluation	All except greedy and IP	~50x pre-iter speedup
Alias Table weighted sampling	Search-related algorithm	O(n) to O(1) exam selection
Feasible first tracking	All	Feasible > infeasible
Periodic full evaluation resyncs	All	Prevent divergence by correcting the delta drifting
Shared greedy initial	All via main.cpp	Compute the construction once, not pre-algo

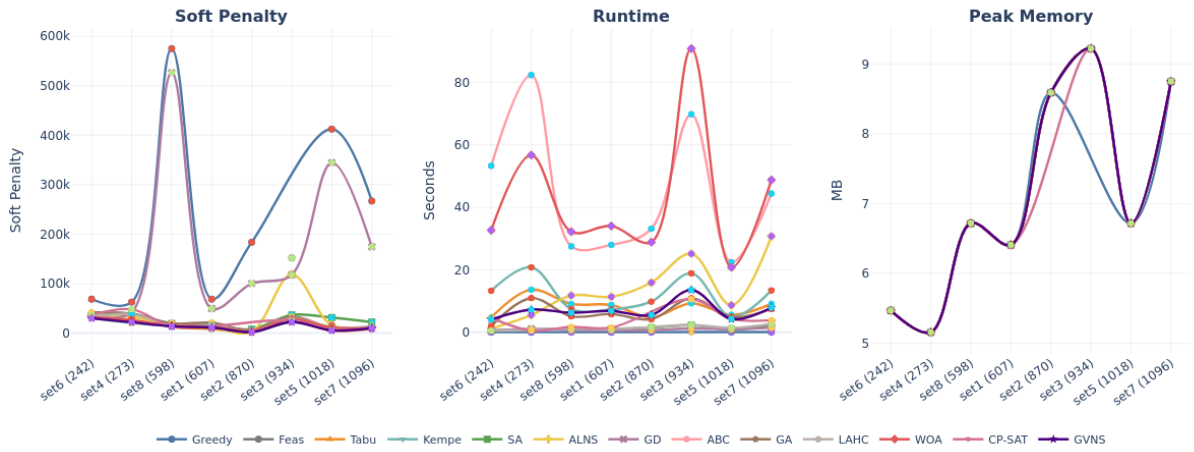
These are just some highlights, and the research paper will go into greater detail on how each strategy was used and its impact on the algorithms' overall performance. We didn't encounter this one in the initial implementation; after careful consideration, we tested several of these implementations to assess their impact on the overall algorithm.

Another thing worth mentioning is that our non-trivial algorithms are hybrid. It includes tight coupling, which shares state and information, and loose coupling, which alternates but doesn't share state. We also tested a sequential pipeline via a chain to see if it performs better than a standalone algorithm. The hybrid algorithm seems useful in our testing, and we're currently working to complete the chain method to see whether it performs better. So far, everything seems promising in our research.



What was researched in our experiment? We track runtime and memory usage in our experiment, along with the number of exams and the soft penalty for each exam. Things we might not track but have considered: the difference between languages, but Python is so slow that we threw that idea out the window. Another was tracking how each parameter tuning impacts the overall performance, which might still be considered if we have time. Aside from everything mentioned, my next step is to write the research paper based on all my findings.

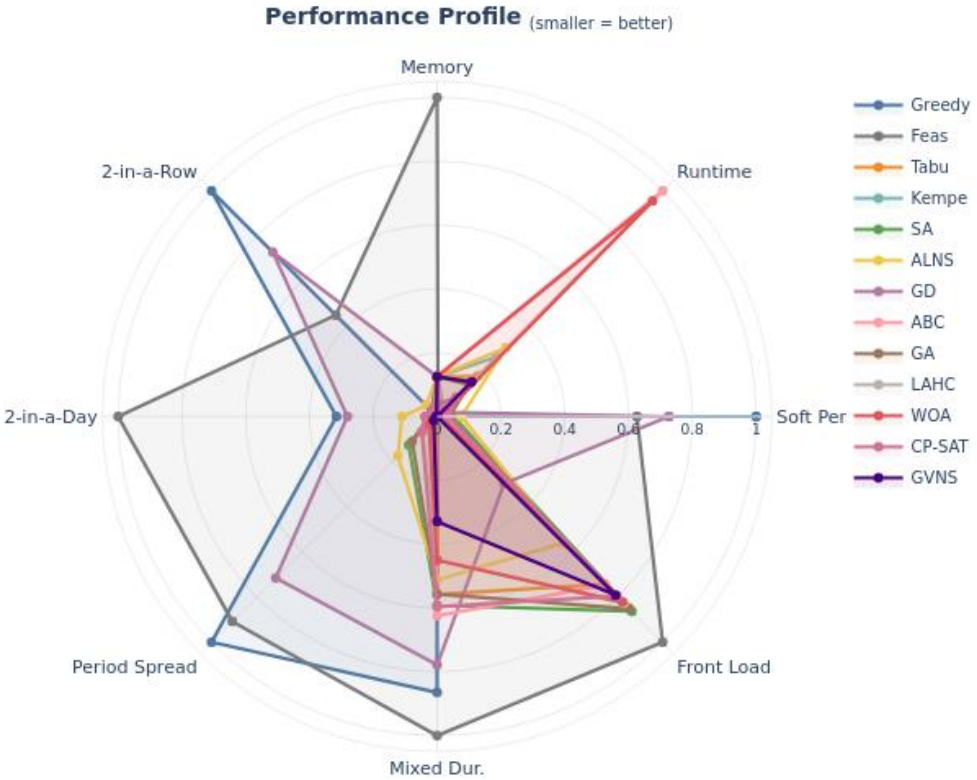




Soft Penalty Heatmap (per-dataset normalised, n/f = not feasible)

	set6	set4	set8	set1	set2	set3	set5	set7
GVNS	30,070	22,065	13,830	11,317	2,239	22,331	5,512	9,955
Kempe	30,860	20,431	14,435	11,403	1,830	22,850	9,021	9,933
ABC	33,690	23,405	14,237	13,609	2,925	24,953	5,527	9,738
Tabu	31,620	38,587	12,861	8,373	2,675	23,699	4,922	8,138
WOA	32,345	26,567	15,014	13,097	3,788	28,296	11,207	10,664
LAHC	32,640	38,354	15,036	15,964	3,520	29,950	12,577	12,827
CP-SAT	36,950	48,197	13,572	14,181	n/f	25,977	6,302	10,661
GA	41,700	39,222	19,597	21,126	5,641	33,268	11,369	13,045
SA	35,010	38,321	19,400	17,029	7,857	36,785	31,400	22,373
ALNS	37,285	32,563	17,384	11,209	3,032	118,586	16,267	13,923
GD	36,830	42,482	526,245	49,693	100,193	117,775	344,718	174,522

Relative
Worst
Mid
Best


Performance Profile (smaller = better)
Memory
Runtime
Soft Per
Front Load
Mixed Dur.
Period Spread
2-in-a-Day
2-in-a-Row
0
0.2
0.4
0.6
0.8
1
Greedy
Feas
Tabu
Kempe
SA
ALNS
GD
ABC
GA
LAHC
WOA
CP-SAT
GVNS